Hazard Analysis
Software Engineering

Team 6, Blockchainerz
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

Table 1: Revision History

Date	**Developer(s)**	**Change**
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **1**
3.1 Components Within the System Boundary 1
3.2 Components Outside the System Boundary 2

4 Critical Assumptions **3**

5 Failure Mode and Effect Analysis **3**

6 Safety and Security Requirements **4**

7 Roadmap **4**

8 Appendix — Generative AI Use Disclosure **5**
8.1 AI Tools Used 5
8.2 How and Where Used 5
8.3 Verification of AI-Generated Content 5

9 Appendix — Reflection **6**

[You are free to modify this template. —SS]

1 Introduction

[You can include your definition of what a hazard is here. —SS]

2 Scope and Purpose of Hazard Analysis

[You should say what **loss** could be incurred because of the hazards. —SS]

3 System Boundaries and Components

This section divides DAIIM into its major components and identifies where the boundary of the system lies. Components inside the boundary are built, deployed, and maintained by the team, and their behaviour can be changed to mitigate hazards. Components outside the boundary are part of DAIIM's environment: DAIIM depends on them, but cannot control them, so hazards originating from them can only be detected, tolerated, or contained. The components below are based on the requirements in the SRS and the technologies listed in the Development Plan. They are preliminary and will be refined in the design documents.

3.1 Components Within the System Boundary

1. **Web Client.** The browser-based user interface, built with React, through which task requesters and inference providers use DAIIM. It connects to the user's wallet for authentication, lets users browse and create tasks, submit inference requests, register as providers, and view results, reputations, stakes, and rewards. Before each transaction, it shows the user the action being performed and the amount of tokens being transferred, and afterwards it shows the transaction's status and identifier. It also performs an initial check that request inputs conform to the task's input schema. *Related SRS requirements:* Accounts and Authorization, Inference Tasks, Inference Requests, Aggregation and Results, Transparency.

2. **On-Chain Programs.** The Solana programs, written in Rust, that hold DAIIM's authoritative state and all of its tokens. They are responsible for:

   - the registry of inference tasks and their parameters (minimum providers, reputation tolerance, acceptance period, maximum prediction time);
   - provider registration and the escrow of provider stakes;
   - escrow of requester payments and the lifecycle of each inference request (acceptance period, prediction phase, cancellation and refunds);

- recording predictions, aggregating them into the final result, and updating provider reputations;
- distributing rewards, and slashing or burning provider stakes.

Because this component holds funds and blockchain transactions cannot be reversed, a defect here can cause direct and permanent financial loss. *Related SRS requirements:* Inference Tasks, Provider Registration and Staking, Provider Acceptance and Predictions, Aggregation and Results, Reputation and Rewards, Penalties, Transparency.

3. **Off-Chain Backend Service.** A Node.js and TypeScript service that performs the work that is impractical or unsafe to do on-chain. It monitors the on-chain programs for new inference requests and broadcasts them to the providers registered for the task, stores request input data and distributes it only to the requester and the providers that accepted the request, and deletes the input data once the request is completed or cancelled. It may also submit the transactions that move a request from one phase to the next when a deadline passes, since on-chain programs cannot act on their own, and it serves marketplace information to the Web Client. Since this service is a single point that the team operates, it must not be able to change results, payments, or reputations, which are controlled only by the on-chain programs. *Related SRS requirements:* Inference Requests, Provider Acceptance and Predictions, Data Handling.

4. **Off-Chain Database.** A PostgreSQL database used by the backend service. It temporarily holds encrypted request input data, which must not be stored on the blockchain, as well as task information that is too large to store on-chain economically (such as problem descriptions, schemas, and references to training data) and indexed copies of on-chain records for fast browsing. The on-chain records remain the source of truth for any data that appears in both places. *Related SRS requirements:* Inference Tasks, Data Handling.

5. **Provider Interface.** The interface, such as an API or client library, through which an inference provider's model receives broadcast requests, accepts requests, retrieves input data, and submits predictions. It checks that predictions conform to the task's output schema before they are submitted. This component ends where the provider's own model begins. *Related SRS requirements:* Provider Acceptance and Predictions.

3.2 Components Outside the System Boundary

1. **Solana Blockchain Network.** The public Solana network, including its validators and the RPC nodes used to read state and submit transactions. DAIIM relies on it to execute the on-chain programs, store records, and provide the time used for request deadlines. It may become congested, raise fees, delay or drop transactions, or suffer outages.

2. **User Wallets.** Third-party Web3 wallets, such as browser wallet extensions, that hold users' private keys and sign transactions. DAIIM cannot recover a lost key, and cannot distinguish a compromised wallet from its legitimate owner.

3. **Inference Providers' Models and Infrastructure.** The machine learning models, computing resources, and network connections of each provider. DAIIM receives only the predictions they produce and cannot inspect the models. Providers may be unavailable, inaccurate, careless, or dishonest, and several providers may collude.

4. **Task Requesters and Their Input Data.** The people who define tasks and submit requests, and the sources of the input data they provide. DAIIM can check that inputs conform to a task's schema, but cannot verify that the data is correct, that task parameters are sensible, or that the data is not sensitive.

5. **Downstream Decisions.** The decisions that task requesters and beneficiaries, such as emergency management agencies, make using DAIIM's results. DAIIM is responsible for providing accurate results and information about their quality, but not for how the results are used.

6. **Hosting Infrastructure and Browsers.** The cloud or server infrastructure that hosts the backend service and database, and the web browsers on which the Web Client runs.

4 Critical Assumptions

[These assumptions that are made about the software or system. You should minimize the number of assumptions that remove potential hazards. For instance, you could assume a part will never fail, but it is generally better to include this potential failure mode. —SS]

5 Failure Mode and Effect Analysis

[Include your FMEA table here. This is the most important part of this document. —SS] [The safety requirements in the table do not have to have the prefix SR. The most important thing is to show traceability to your SRS. You might trace to requirements you have already written, or you might need to add new requirements. —SS] [If no safety requirement can be devised, other mitigation strategies can be entered in the table, including strategies involving providing additional documentation, and/or test cases. —SS]

6 Safety and Security Requirements

[Newly discovered requirements. These should also be added to the SRS. (A rationale design process how and why to fake it.) —SS]

7 Roadmap

[Which safety requirements will be implemented as part of the capstone timeline? Which requirements will be implemented in the future? —SS]

8 Appendix — Generative AI Use Disclosure

This section provides an overview of how generative AI tools were used in the making of this document.

8.1 AI Tools Used

- Claude Opus 5.5 (Anthropic), used through Claude Code.

8.2 How and Where Used

Claude Opus 5.5 was used throughout this document to help ideate and draft its content. The team provided the tool with the SRS and the Development Plan, and used it to brainstorm the hazards, components, assumptions, and requirements that the document should address, and to produce first drafts of the text. In Section 3 (System Boundaries and Components), for example, it proposed a division of DAIIM into components inside and outside the system boundary, drafted the description of each component, and linked each component to the groups of functional requirements in the SRS that it is responsible for. It was also used to draft the text of this appendix.

The tool was used as an aid, not as a replacement for the team's own work. Team members were involved in every sentence of the document: each AI-assisted draft was read, discussed, and edited by the team, and ideas that did not fit the project were discarded or rewritten.

8.3 Verification of AI-Generated Content

Every sentence in this document was reviewed by at least one team member before it was included. AI-assisted content was checked against the SRS and the Development Plan to confirm that it describes responsibilities and behaviour that the requirements actually assign to DAIIM, that it does not introduce requirements or design decisions the team has not agreed to, and that it is consistent with the technologies the Development Plan lists. References to SRS requirements were checked by hand. Content that is still undecided, such as how request phases are advanced when deadlines pass, is marked as preliminary and will be confirmed in the design documents. The team remains responsible for the accuracy, completeness, and consistency of this document, regardless of whether material was first drafted by a team member or with the help of an AI tool.

9 Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?
4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?